



What is cc really doing - the condensed version

We understand how `cc` works in its simplest form:

- we invoke `cc` on a single C source file,
- we know the C-processor is invoked to include system-wide header files, and to define our own preprocessor and definitions macros,
- the output of the preprocessor becomes the input of the "true" compiler,
- the output of the compiler (for correct programs!) is an executable program (and we may use the `-o` option to provide a specific executable name).

What is cc really doing - the long version

Not surprisingly, there's much more going on!

`cc` is really a front-end program to a number of *passes* or *phases* of the whole activity of "converting" our C source files to executable programs:

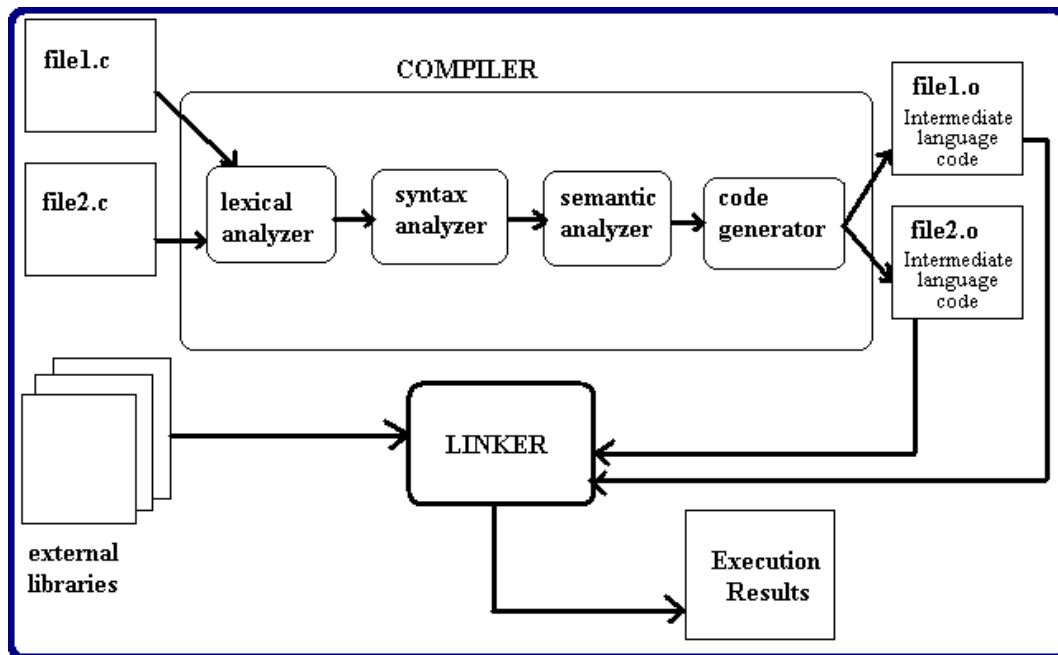
1. *foreach C source file we're compiling:*
 - i. the C source code is given to the *C preprocessor*,
 - ii. the C preprocessor's output is given to the *C parser*,
 - iii. the parser's output is given to a *code generator*,
 - iv. the code generator's output is given to a *code optimizer*,
 - v. the code optimizer's output, termed *object code*, is written to a disk file termed an *object file*,
2. all necessary object files (there may be more than one, and some may be standard C libraries, operating system-specific, or provided by a third-party), are presented to a program named the *linker*, to be "combined" together, and
3. the linker's output is written to disk as an *executable file*.

CITS2002 Systems Programming, Lecture 12, p1, 6th September 2019.

What is cc really doing - in a picture

Additional details:

- `cc` determines which compilation phases to perform based on the command-line options and the file name extensions provided.
- The compiler passes object files (with the filename suffix `.o`) and any unrecognized file names to the *linker*.
- The linker then determines whether files are object files or library files (often with the filename suffix `.a`).
- The linker combines all required *symbols* (e.g. your *main()* function from your `.o` file and the *printf()* function from C's standard library) to form the single executable program file.



CITS2002 Systems Programming, Lecture 12, p2, 6th September 2019.

Developing larger C programs in multiple files

Just as C programs should be divided into a number of functions (we often say the program is *modularized*), larger C programs should be divided into multiple source files.

The *motivations* for using multiple source files are:

- each file (often containing multiple related functions) may perform (roughly) a single role,
- the number of unnecessary global variables can be significantly reduced,
- we may easily edit the multiple files in separate windows,
- large projects may be undertaken by multiple people each working on a subset of the files,
- each file may be separately compiled into a distinct object file,
- small changes to one source file do not require all other source files to be recompiled.

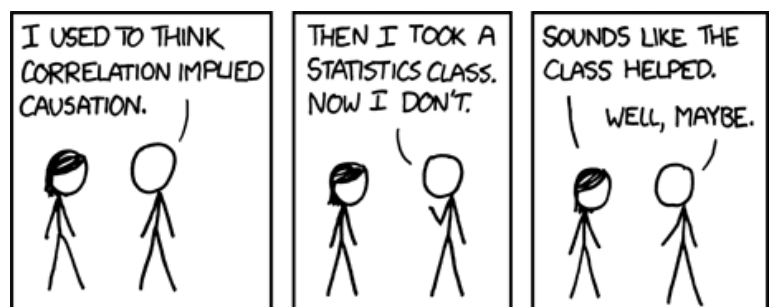
All object files are then linked to form a single executable program.

CITS2002 Systems Programming, Lecture 12, p3, 6th September 2019.

A simple multi-file program

For this lecture we'll develop a simple project to calculate the correlation of [some student marks](#), partitioned into multiple files. The input data file contains two columns of marks - from a project marked out of 40, and an exam marked out of 60.

- [calcmarks.h](#) - contains globally visible declarations of types, functions, and variables
- [calcmarks.c](#) - contains `main()`, checks arguments, calls functions
- [globals.c](#) - defines global variables required by all files
- [readmarks.c](#) - performs all datafile reading
- [correlation.c](#) - performs calculations



Each C file depends on a *common header file*, which we will name `calcmarks.h`.

Providing declarations in header files

We employ the shared header file, *calcmarks.h*, to *declare* the program's:

- C preprocessor constants and macros,
- globally visible functions (may be called from other files), and
- globally visible variables (may be accessed/modified from all files).

The header file is used to *announce* their existence using the **extern** keyword.

The header file does not actually provide function implementations (code) or allocate any memory space for the variables.

```
#include <stdio.h>
#include <stdbool.h>
#include <math.h>

// DECLARE GLOBAL PREPROCESSOR CONSTANTS
#define MAXMARKS 200

// DECLARE GLOBAL FUNCTIONS
extern int      readmarks(FILE *); // parameter is not named
extern void     correlation(int);  // parameter is not named

// DECLARE GLOBAL VARIABLES
extern double   projmarks[];      // array size is not provided
extern double   exammarks[];     // array size is not provided

extern bool     verbose;          // declarations do not provide initializations
```

Notice that, although we have indicated that function *readmarks()* accepts one *FILE ** parameter, we have not needed to give it a name.

Similarly, we have declared the existence of arrays, but have not indicated/provided their sizes.

Providing our variable definitions

In the C file *globals.c* we finally *define* the global variables.

It is here that the compiler allocates memory space for them.

In particular, we now define the size of the *projmarks* and *exammarks* arrays, in a manner dependent on the preprocessor constants from *calcmarks.h*

This allows us to provide all configuration information in one (or more) header files. Other people modifying your programs, in years to come, will know to look in the header file(s) to adjust the constraints of your program.

```
#include "calcmarks.h" // we use double-quotes

double   projmarks[ MAXMARKS ]; // array's size is defined
double   exammarks[ MAXMARKS ]; // array's size is defined

bool     verbose = false;        // global is initialized
```

Global variables are automatically 'cleared'

By default, global variables are initialized by filling them with zero-byte patterns.

This is convenient (of course, it's by design) because the zero-byte pattern sets the variables (scalars and arrays) to:

- 0 (for **ints**),
- '\0' (for **chars**),
- 0.0 (for **floats** and **doubles**),
- false (for **bools**), and
- zeroes (for pointers).

Note that we could have omitted the initialisation of *verbose* to *false*, but providing an explicit initialisation is much clearer.

CITS2002 Systems Programming, Lecture 12, p6, 6th September 2019.

The *main()* function

All of our C source files now include our local header file. Remembering that file inclusion simply "pulls in" the textual content of the file, our C files are now provided with the *declarations* of all global functions and global variables.

Thus, our code may now call global functions, and access global variables, without (again) declaring their existence:

```
#include "calcmarks.h"    // local header file provides declarations

int main(int argc, char *argv[])
{
    int nmarks = 0;

    // IF WE RECEIVED NO COMMAND-LINE ARGUMENTS, READ THE MARKS FROM stdin
    if(argc == 1)
    {
        nmarks += readmarks(stdin);
    }
    // OTHERWISE WE ASSUME THAT EACH COMMAND-LINE ARGUMENT IS A FILE NAME
    else
    {
        for(int a=1 ; a<argc ; ++a)
        {
            FILE *fp = fopen(argv[a], "r");

            if(fp == NULL)
            {
                printf("Cannot open %s\n", argv[a]);
                exit(EXIT_FAILURE);
            }
            nmarks += readmarks(fp);
        }
        // CLOSE THE FILE THAT WE OPENED
        fclose(fp);
    }
    // IF WE RECEIVED SOME MARKS, REPORT THEIR CORRELATION
    if(nmarks > 0)
    {
        correlation(nmarks);
    }
    return 0;
}
```

In the above function, we have used to a local variable, *nmarks*, to maintain a value (both receiving it from function calls, and passing it to other functions).

nmarks could have been another global variable but, generally, we strive to minimize the number of globals.

CITS2002 Systems Programming, Lecture 12, p7, 6th September 2019.

Reading the marks from a file

Nothing remarkable in this file:

```

#include "calcmarks.h"    // local header file provides declarations

int readmarks(FILE *fp)
{
    char    line[BUFSIZ];
    int     nmarks = 0;

    double  thisproj;
    double  thisexam;

    ....

    // READ A LINE FROM THE FILE, CHECKING FOR END-OF-FILE OR AN ERROR
    while( fgets(line, sizeof line, fp) != NULL )
    {

    // WE'RE ASSUMING THAT WE LINE PROVIDES TWO MARKS
        ....    // get 2 marks from this line

        projmarks[ nmarks ] = thisproj;    // update global array
        exammarks[ nmarks ] = thisexam;

        ++nmarks;

        if(verbose)    // access global variable
        {
            printf("read student %i\n", nmarks);
        }
    }
    return nmarks;
}

```

CITS2002 Systems Programming, Lecture 12, p8, 6th September 2019.

Calculate the correlation coefficient (the least exciting part)

```

#include "calcmarks.h"    // local header file provides declarations

void correlation(int nmarks)
{
    // MANY LOCAL VARIABLES REQUIRED TO CALCULATE THE CORRELATION
    double  sumx   = 0.0;
    double  sumy   = 0.0;
    double  sumxx  = 0.0;
    double  sumyy  = 0.0;
    double  sumxy  = 0.0;

    double  ssxx, ssyy, ssxy;
    double  r, m, b;

    // ITERATE OVER EACH MARK
    for(int n=0 ; n < nmarks ; ++n)
    {
        sumx   += projmarks[n];
        sumy   += exammarks[n];
        sumxx  += (projmarks[n] * projmarks[n]);
        sumyy  += (exammarks[n] * exammarks[n]);
        sumxy  += (projmarks[n] * exammarks[n]);
    }

    ssxx  = sumxx - (sumx*sumx) / nmarks;
    ssyy  = sumyy - (sumy*sumy) / nmarks;
    ssxy  = sumxy - (sumx*sumy) / nmarks;

    // CALCULATE THE CORRELATION COEFFICIENT, IF POSSIBLE
    if((ssxx * ssyy) == 0.0)
    {
        r   = 1.0;
    }
}

```

```

    }
    else
    {
        r    = ssxy / sqrt(ssxx * ssyy);
    }
    printf("correlation is %.4f\n", r);

// DETERMINE THE LINE OF BEST FIT, IT ONE EXISTS
if(ssxx != 0.0)
{
    m    = ssxy / ssxx;
    b    = (sumy / nmarks) - (m*(sumx / nmarks));
    printf("line of best fit is y = %.4fx + %.4f\n", m, b);
}
}

```

CITS2002 Systems Programming, Lecture 12, p9, 6th September 2019.

Maintaining multi-file projects

As large projects grow to involve many, tens, even *hundreds*, of source files, it becomes a burden to remember which ones have been recently changed and, hence, need recompiling.

This is particularly difficult to manage if multiple people are contributing to the same project, each editing different files.

As an easy way out, we could (expensively) just compile everything!

```
cc -std=c99 -Wall -pedantic -Werror -o calcmarks calcmarks.c globals.c readmarks.c correlation.c
```

Introducing *make*

The program *make* maintains up-to-date versions of programs that result from a sequence of actions on a set of files.

make reads specifications from a file typically named *Makefile* or *makefile* and performs the actions associated with *rules* if indicated files are "out of date".

Basically, in pseudo-code (*not* in C) :

```

if (files on which a certain file depends)
    i) do not exist, or
    ii) are not up-to-date
then
    create an up-to-date version;

```

make operates over rules and actions recursively and will abort its execution if it cannot create an up-to-date file on which another file depends.

Note that *make* can be used for many tasks other than just compiling C - such as compiling other code from programming languages, reformatting text and web documents, making backup copies of files that have recently changed, etc.

CITS2002 Systems Programming, Lecture 12, p10, 6th September 2019.

Dependencies between files

From our pseudo-code:

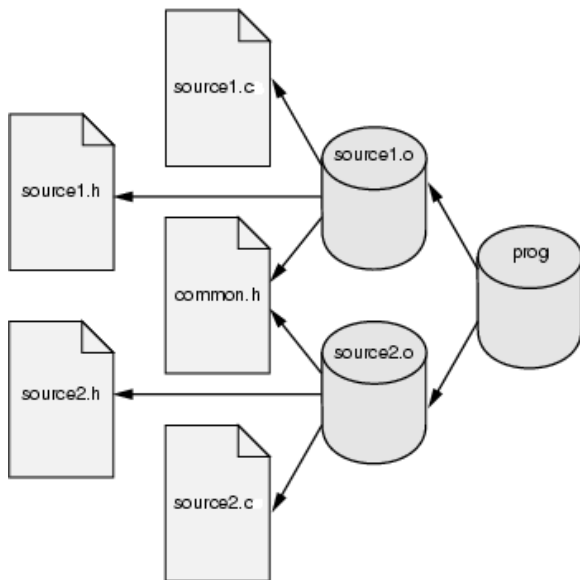
```
if (files on which a certain file depends)
```

```

    i) do not exist, or
    ii) are not up-to-date
then
    create an up-to-date version;

```

we are particularly interested in the *dependencies* between various files - certain files *depend* on others and, if one changes, it triggers the "rebuilding" of others:



So:

- The executable program *prog* is dependent on one or more object files (*source1.o* and *source2.o*).
 - Each object file is (typically) dependent on one C source file (suffix *.c*) and, often, on one or more header files (suffix *.h*).
- So:
- If a header file or a C source file are modified (edited), then an object file needs rebuilding (by *cc*).
 - If one or more object files are rebuilt or modified (by *cc*), then the executable program need rebuilding (by *cc*).

NOTE that the source code files (suffix *.c*) are not dependent on the header files (suffix *.h*).

CITS2002 Systems Programming, Lecture 12, p11, 6th September 2019.

A simple *Makefile* for our program

For the case of our multi-file program, *calcmarks*, we can develop a very verbose *Makefile* which fully describes the actions required to compile and link our project files.

```

# A Makefile to build our 'calcmarks' project

calcmarks : calcmarks.o globals.o readmarks.o correlation.o
— tab →cc -std=c99 -Wall -pedantic -Werror -o calcmarks \
    calcmarks.o globals.o readmarks.o correlation.o -lm

calcmarks.o : calcmarks.c calcmarks.h
— tab →cc -std=c99 -Wall -pedantic -Werror -c calcmarks.c

globals.o : globals.c calcmarks.h
— tab →cc -std=c99 -Wall -pedantic -Werror -c globals.c

readmarks.o : readmarks.c calcmarks.h
— tab →cc -std=c99 -Wall -pedantic -Werror -c readmarks.c

correlation.o : correlation.c calcmarks.h
— tab →cc -std=c99 -Wall -pedantic -Werror -c correlation.c

```

[download this *Makefile*.](#)

Of note:

- each *target*, at the beginning of lines, is followed by the *dependencies* (typically other files) on which it depends,
- each target may also have one or more *actions* that are performed/executed if the target is out-of-date with respect to its dependencies,
- actions must commence with the **tab character**, and

- each (line) is passed verbatim to a *shell* for execution - just as if you would type it by hand. Very long lines may be split using the backslash character.

CITS2002 Systems Programming, Lecture 12, p12, 6th September 2019.

Variable substitutions in *make*

As we see from the previous example, *Makefiles* can themselves become long, detailed files, and we'd like to "factor out" a lot of the common information.

It's similar to setting *constants* in C, with *#define*

Although not a full programming language, *make* supports simple variable definitions and variable substitutions (and even functions!).

```
# A Makefile to build our 'calcmarks' project

C99      = cc -std=c99
CFLAGS   = -Wall -pedantic -Werror

calcmarks : calcmarks.o globals.o readmarks.o correlation.o
            $(C99) $(CFLAGS) -o calcmarks \
                calcmarks.o globals.o readmarks.o correlation.o -lm

calcmarks.o : calcmarks.c calcmarks.h
            $(C99) $(CFLAGS) -c calcmarks.c

globals.o : globals.c calcmarks.h
            $(C99) $(CFLAGS) -c globals.c

readmarks.o : readmarks.c calcmarks.h
            $(C99) $(CFLAGS) -c readmarks.c

correlation.o : correlation.c calcmarks.h
            $(C99) $(CFLAGS) -c correlation.c
```

Of note:

- variables are usually defined near the top of the *Makefile*.
- the variables are simply expanded in-line with \$(VARNAME).
- **warning** - the syntax of *make*'s variable substitutions is slightly different to those of our standard shells.

CITS2002 Systems Programming, Lecture 12, p13, 6th September 2019.

Variable substitutions in *make*, *continued*

As our projects grow, we add more C source files to the project. We should *refactor* our *Makefiles* when we notice common patterns:

```
# A Makefile to build our 'calcmarks' project

PROJECT = calcmarks
HEADERS = $(PROJECT).h
OBJ      = calcmarks.o globals.o readmarks.o correlation.o

C99      = cc -std=c99
CFLAGS   = -Wall -pedantic -Werror

$(PROJECT) : $(OBJ)
```



```

$(C99) $(CFLAGS) -o $(PROJECT) $(OBJ) -lm

calcmkmarks.o : calcmkmarks.c $(HEADERS)
$(C99) $(CFLAGS) -c calcmkmarks.c

globals.o : globals.c $(HEADERS)
$(C99) $(CFLAGS) -c globals.c

readmarks.o : readmarks.c $(HEADERS)
$(C99) $(CFLAGS) -c readmarks.c

correlation.o : correlation.c $(HEADERS)
$(C99) $(CFLAGS) -c correlation.c

clean:
rm -f $(PROJECT) $(OBJ)

```

Of note:

- we have introduced a new variable, `PROJECT`, to name our project,
- the value of the new variable, `HEADERS` is defined by accessing the value of `$(PROJECT)`,
- we have introduced a new variable, `OBJ`, to collate all of our object files,
- our project specifically depends on our object files,
- we have a new target, named *clean*, to remove all unnecessary files. *clean* has no dependencies, and so will always be executed if requested.

CITS2002 Systems Programming, Lecture 12, p14, 6th September 2019.

Employing automatic variables in a *Makefile*

We further note that each of our object files depends on its C source file, and that it would be handy to reduce these very common lines.

make provides a (wide) variety of filename patterns and automatic variables to considerably simplify our actions:

```

# A Makefile to build our 'calcmkmarks' project

PROJECT = calcmkmarks
HEADERS = $(PROJECT).h
OBJ      = calcmkmarks.o globals.o readmarks.o correlation.o

C99      = cc -std=c99
CFLAGS   = -Wall -pedantic -Werror

$(PROJECT) : $(OBJ)
$(C99) $(CFLAGS) -o $(PROJECT) $(OBJ) -lm

%.o : %.c $(HEADERS)
$(C99) $(CFLAGS) -c $<

clean:
rm -f $(PROJECT) $(OBJ)

```

Of note:

- the pattern `%.o` matches, in turn, each of the 4 object filenames to be considered,
- the pattern `%.c` is "built" from the C file corresponding to the `%.o` file,
- the automatic variable `$<` is "the reason we're here", and

- the linker option `-lm` indicates that our project requires something from C's standard *maths* library (`sqrt()`).

make supports [many automatic variables](#), which it "keeps up to date" as its execution proceeds:

<code>\$@</code>	This will always expand to the current target.
<code>\$<</code>	The name of the first dependency. This is the first item listed after the colon.
<code>\$?</code>	The names of all the dependencies that are newer than the target.

Fortunately, we rarely need to remember all of these patterns and variables, and generally just copy and modify existing *Makefiles*.